

Diseño de la Arquitectura Full Stack

Backend, API y Servidor



¿Qué es la Arquitectura Full Stack?



La arquitectura Full Stack define cómo se estructura una aplicación completa, desde el frontend que interactúa con usuarios hasta el backend que procesa datos y la base de datos que los almacena.

Un diseño robusto garantiza escalabilidad, seguridad y mantenibilidad a largo plazo.

El Backend: Corazón de la Aplicación



Gestión de Datos

Procesa, almacena y recupera información de forma eficiente y segura



Seguridad

Implementa autenticación, autorización y protección contra amenazas



Lógica de Negocio

Ejecuta las reglas y procesos específicos de la aplicación



Integración

Conecta con servicios externos, APIs y sistemas de terceros

Monolítica vs Microservicios

Arquitectura Monolítica

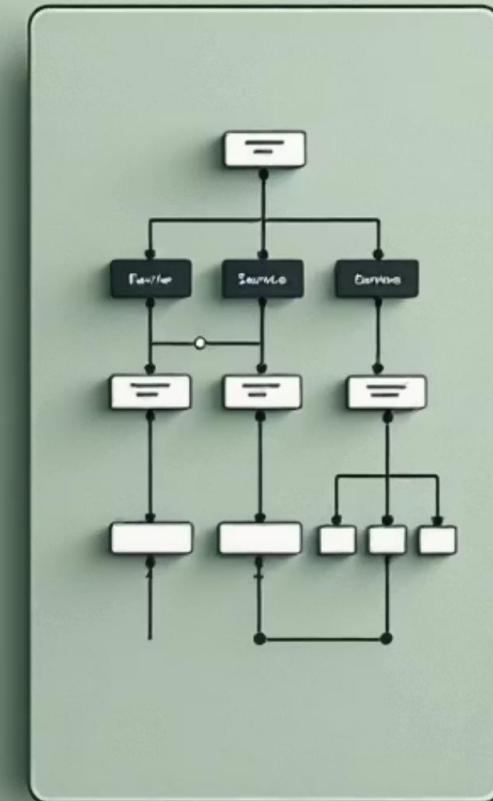
Ventajas: Simple de desarrollar, desplegar y mantener. Ideal para proyectos pequeños o MVPs.

Desventajas: Difícil de escalar, actualizaciones requieren deploy completo, menor flexibilidad tecnológica.

Arquitectura de Microservicios

Ventajas: Escalabilidad horizontal, actualizaciones independientes, tecnologías heterogéneas.

Desventajas: Complejidad en coordinación, mayor overhead de red, desafíos de debugging.



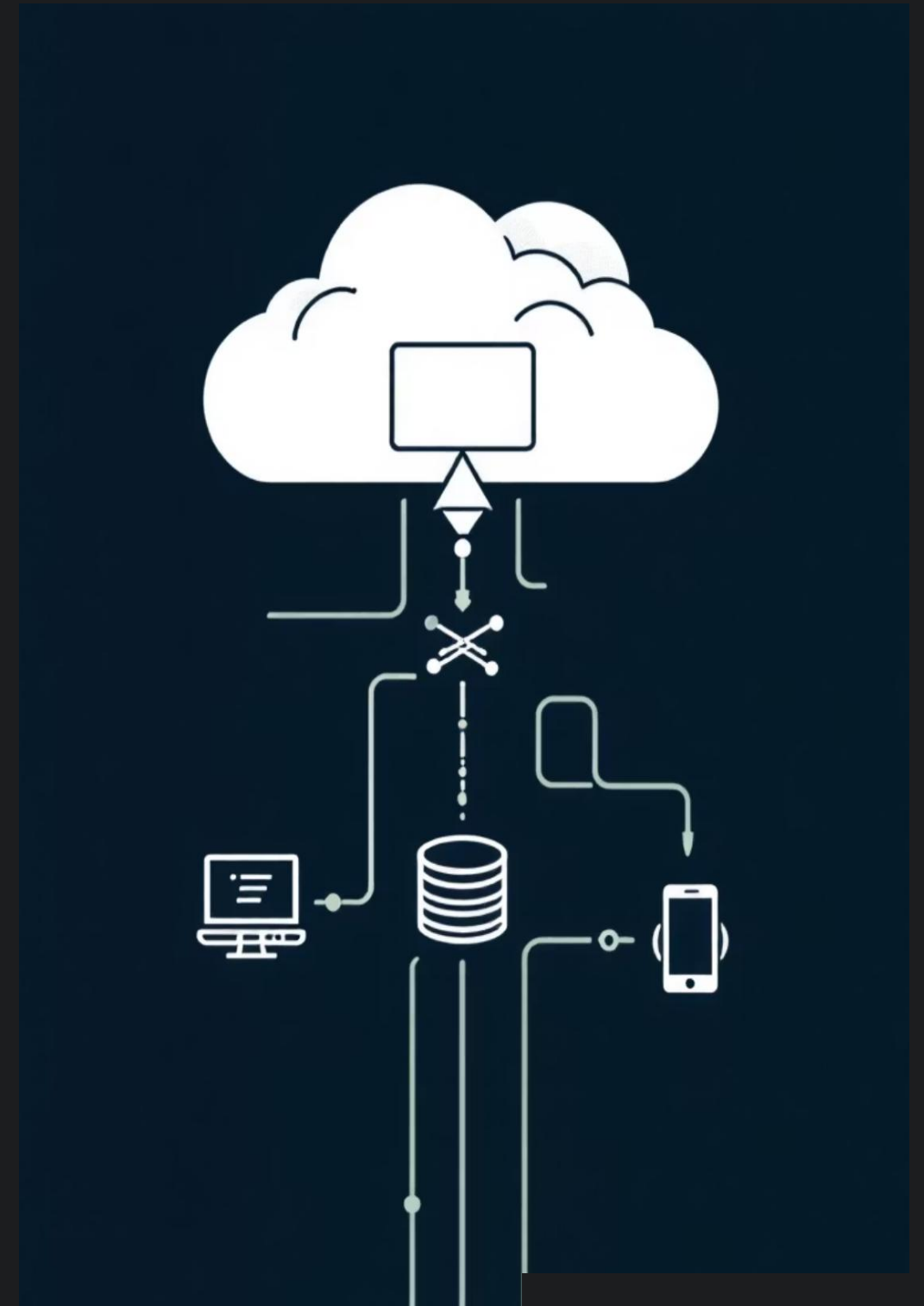
Diseño de APIs RESTful

¿Qué es una API?

Una Interface de Programación de Aplicaciones (API) permite que diferentes sistemas se comuniquen entre sí mediante endpoints definidos.

Métodos HTTP Comunes

- GET - Recuperar datos
- POST - Crear nuevos recursos
- PUT/PATCH - Actualizar existentes
- DELETE - Eliminar recursos



Buenas Prácticas en APIs

01	02	03
Nomenclatura Consistente	Autenticación JWT	Códigos de Estado HTTP
Usa recursos en plural, verbos en endpoints para acciones específicas	Implementa tokens para verificar identidad y permisos de usuarios	Retorna códigos apropiados: 200 OK, 404 Not Found, 500 Error
04	05	
Documentación Clara	Limitación de Tasa (Rate Limiting)	
Documenta endpoints, parámetros, respuestas y ejemplos de uso	Protege contra ataques de fuerza bruta y sobrecarga del servidor	

Estructura del Servidor



Controllers

Gestiona las solicitudes HTTP entrantes, valida datos y coordina la lógica de negocio



Routes

Define los endpoints URL y enruta las solicitudes a los controllers correspondientes



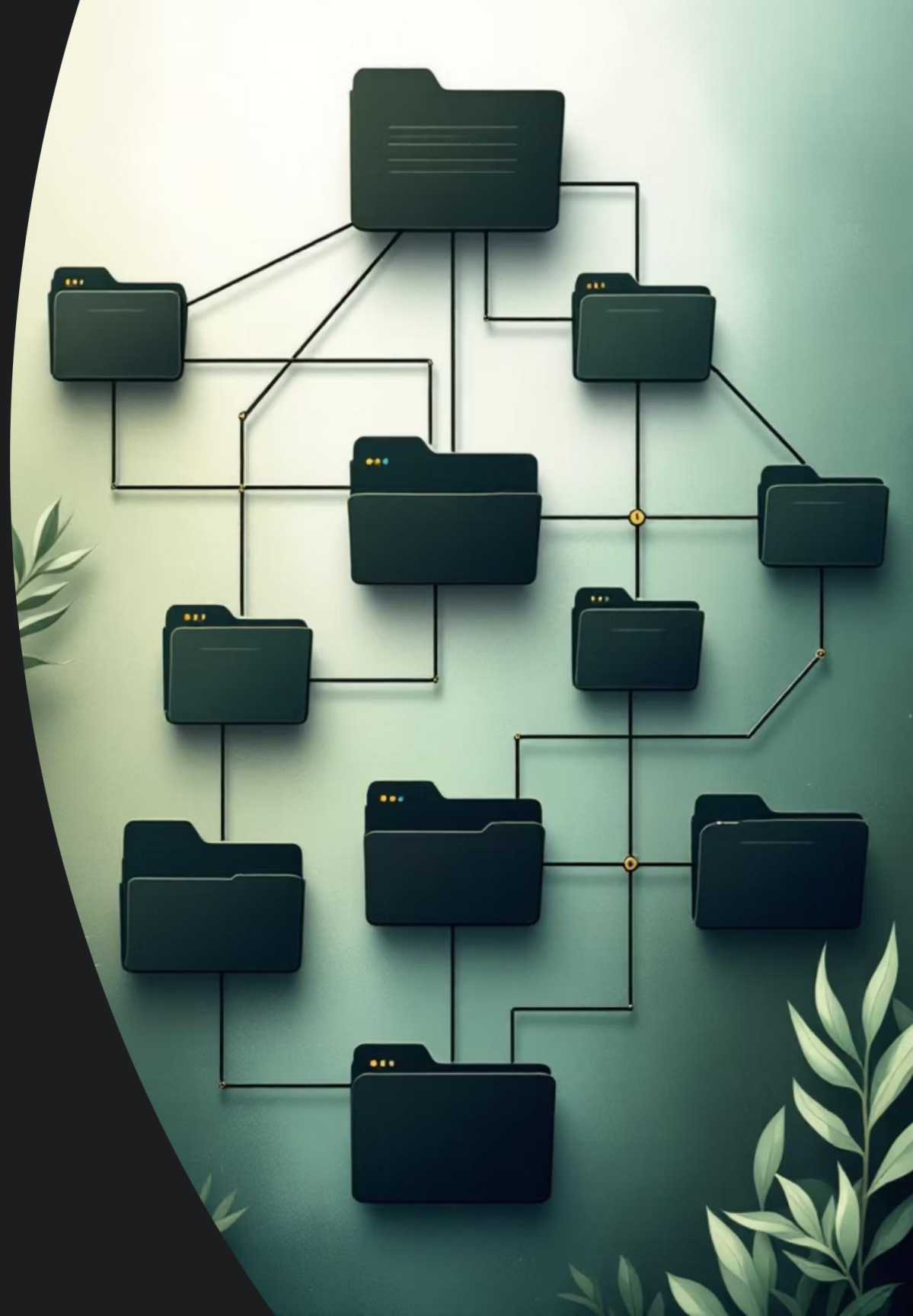
Models

Representa la estructura de datos y define relaciones con la base de datos



Services

Contiene la lógica de negocio compleja y opera de forma independiente de controllers



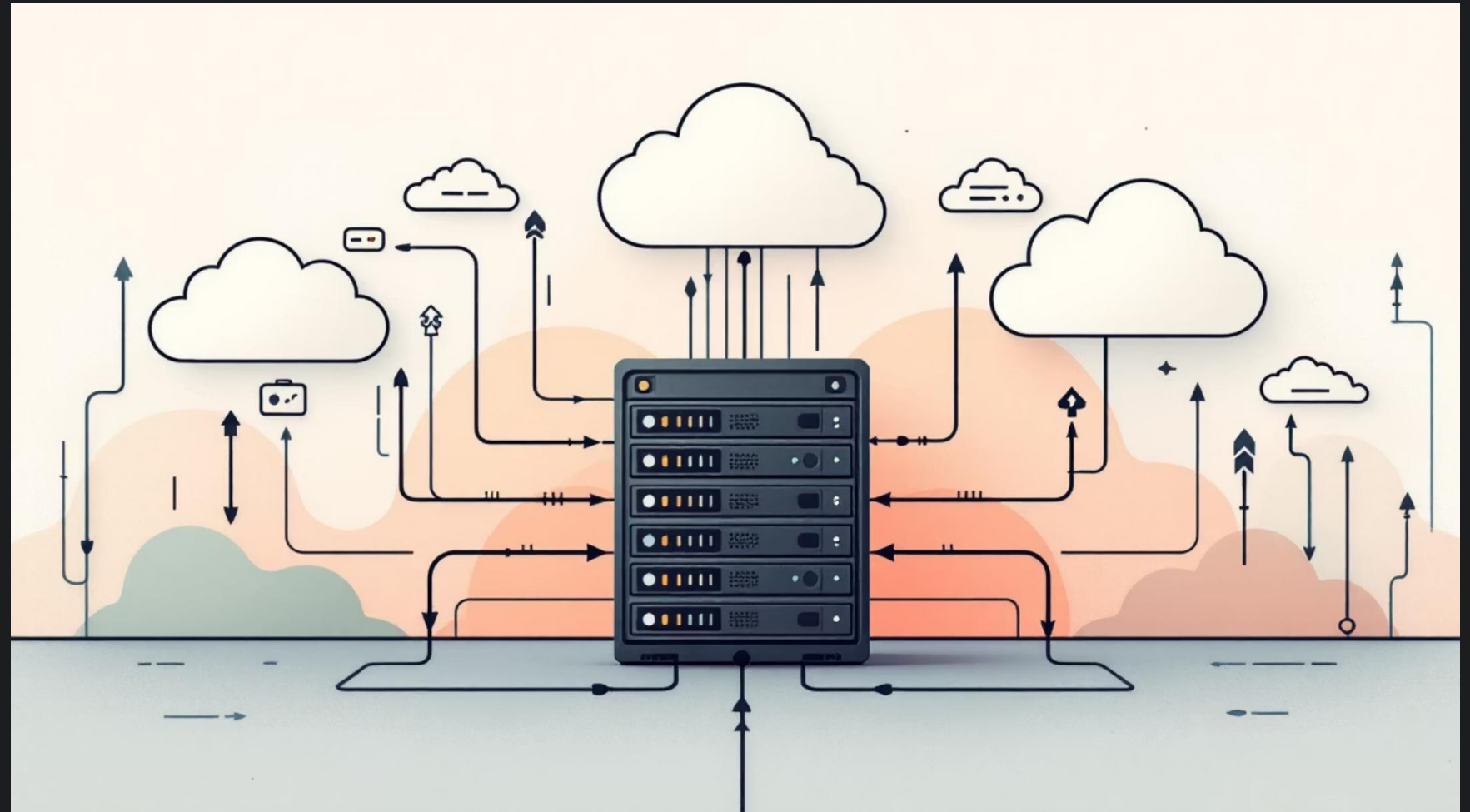
Deploy: Poniendo tu Aplicación en Producción

¿Qué es el Deploy?

El despliegue (deploy) es el proceso de publicar tu código en servidores para que esté accesible al público. Incluye compilación, configuración de entorno y puesta en marcha del servicio.

Proceso Típico

1. Compilación del código
2. Configuración de variables de entorno
3. Migración de base de datos
4. Inicio del servidor



Railway vs Dokploy

Railway

Ventajas: Interfaz intuitiva, configuración automática, integración con GitHub, gratuito para proyectos pequeños.

Uso: Conecta tu repositorio, Railway detecta automáticamente el tipo de proyecto y configura todo.

Dokploy

Ventajas: Control total sobre infraestructura, auto-hospedado, mayor flexibilidad de configuración.

Uso: Instala en tu servidor, configura manualmente proyectos, gestiona recursos directamente.

Comparación Rápida

Aspecto	Railway	Dokploy
Facilidad	Alta (UI intuitiva)	Media (requiere configuración)
Control	Limitado	Completo
Costo Inicial	Gratis (límite)	Requiere servidor propio
Aprendizaje	Rápido	Gradual

Conclusiones Clave



Diseño Fundacional

Elige arquitectura apropiada (monolítica o microservicios) según escala del proyecto



APIs Robustas

Implementa RESTful con buenas prácticas: autenticación, documentación y códigos de estado



Estructura Clara

Organiza código en controllers, routes, models y services para mantenibilidad



Deploy Estratégico

Elige herramienta según necesidades: Railway para facilidad, Dokploy para control total

